

Tarefa para Engenheiro Backend Java

Caso de Uso: Sistema de Reserva de Livros da Biblioteca

Objetivo

Você foi encarregado de projetar e implementar um Serviço de Reserva de Livros para um sistema de biblioteca digital moderno. O serviço deve permitir que os usuários reservem livros, gerenciem suas reservas e garantam que as regras de negócio sejam seguidas corretamente.

Requisitos

API do Serviço de Reservas

Desenvolva uma API RESTful que permita aos usuários:

- Reservar um livro.
- Recuperar uma reserva existente pelo ID.
- Recuperar todas as reservas de um usuário específico.
- Cancelar uma reserva.

Restrições

- Um usuário **não pode reservar mais de 3 livros ao mesmo tempo**.
- Um livro **só pode ser reservado se estiver disponível**.

Esquema de Banco de Dados

Projete um esquema relacional para armazenar os detalhes das reservas.

As reservas devem armazenar:

- O usuário que fez a reserva.
- O livro reservado.
- A data de criação da reserva.
- O status atual da reserva (por exemplo: ativa, cancelada, expirada).

Os livros devem armazenar:

- Título, autor e ISBN do livro.
- Número de cópias disponíveis para reserva.

Regras de Negócio

- Impedir que um usuário exceda o limite máximo de reservas.
- Atualizar o número de cópias disponíveis do livro ao fazer ou cancelar uma reserva.

Stack Tecnológico

- Java (Spring Boot)

- ORM ou não, fica a seu critério
- Banco de dados relacional de sua escolha

Bônus

Melhorias Lógicas

- As reservas expiram após 7 dias se não forem retiradas.
- O usuário pode visualizar o histórico de reservas, incluindo as expiradas.

Melhorias Técnicas

- Implementar cache usando Redis para otimizar dados acessados com frequência.
- Adicionar testes unitários para as camadas de serviço e repositório.
- Dockerizar a aplicação.
- Implementar um job agendado para expirar reservas após 7 dias.

Critérios de Avaliação

- **Qualidade do Código:** legibilidade, estrutura e manutenibilidade.
- **Design do Banco de Dados:** esquema adequado e uso de índices.
- **Boas Práticas de Microserviços:** arquitetura limpa e separação de responsabilidades.
- **Design da API:** princípios RESTful, tratamento de erros e estrutura de resposta.
- **Considerações de Performance:** consultas eficientes e tratamento de dados.
- **Testes:** presença de testes unitários.

Diretrizes de Entrega

Após concluir a tarefa, pedimos gentilmente que envie o projeto em um arquivo **.ZIP** para a equipe de Aquisição de Talentos como anexo de e-mail. Certifique-se de **incluir apenas o seu trabalho e não incluir bibliotecas relacionadas ou binários compilados**, pois isso pode impedir que o anexo seja transmitido com sucesso.